

Motor de Streaming Distribuido – Documentación Técnica

Sistema de procesamiento distribuido de eventos en tiempo real inspirado en Apache Flink, implementado en Rust con arquitectura master-worker.

Tabla de Contenidos

- [Descripción General](#)
 - [Arquitectura del Sistema](#)
 - [Componentes Principales](#)
 - [Características Implementadas](#)
 - [Guía de Inicio Rápido](#)
 - [Operadores Soportados](#)
 - [API Reference](#)
 - [Testing y Validación](#)
 - [Rendimiento y Benchmarks](#)
 - [Mejores Prácticas](#)
-

Descripción General

Este proyecto implementa un motor de procesamiento de eventos en tiempo real completamente funcional con capacidades de procesamiento distribuido, tolerancia a fallos y agregaciones con ventanas temporales.

Características Principales

- **Procesamiento Distribuido:** Arquitectura master-worker con balanceo de carga automático mediante algoritmo round-robin consciente de capacidad
- **Tolerancia a Fallos:** Detección de fallos mediante heartbeats y re-planificación automática de topologías
- **Ventanas Temporales:** Soporte para ventanas tumbling y sliding con agregaciones (count, sum, average)
- **Checkpointing:** Persistencia periódica del estado de ventanas para auditoría y recuperación

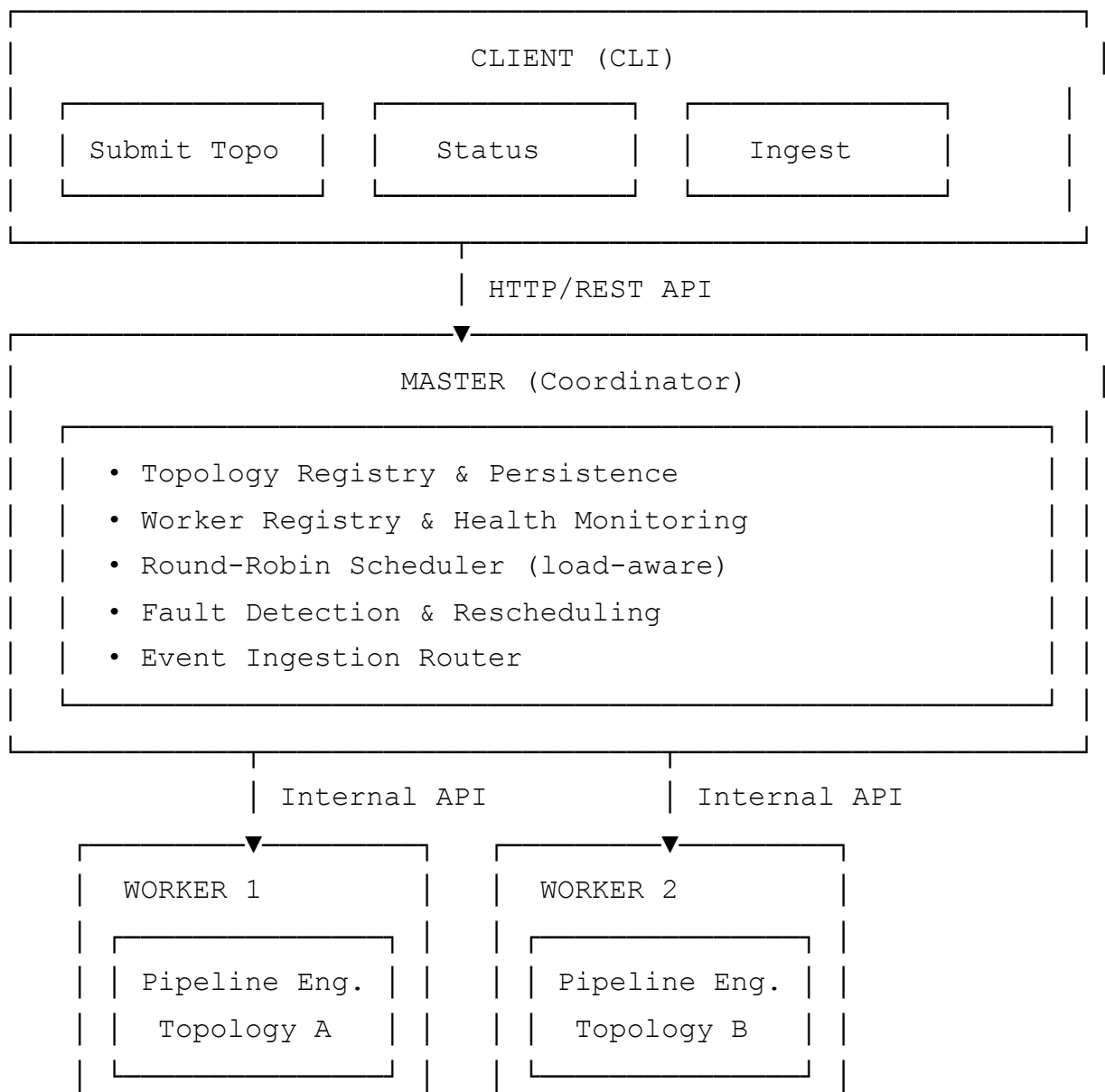
- **Observabilidad:** Métricas en tiempo real de CPU, memoria, throughput y backlog por worker
- **Persistencia:** Estado del master persiste en `state/master_state.json` para recuperación tras reinicios

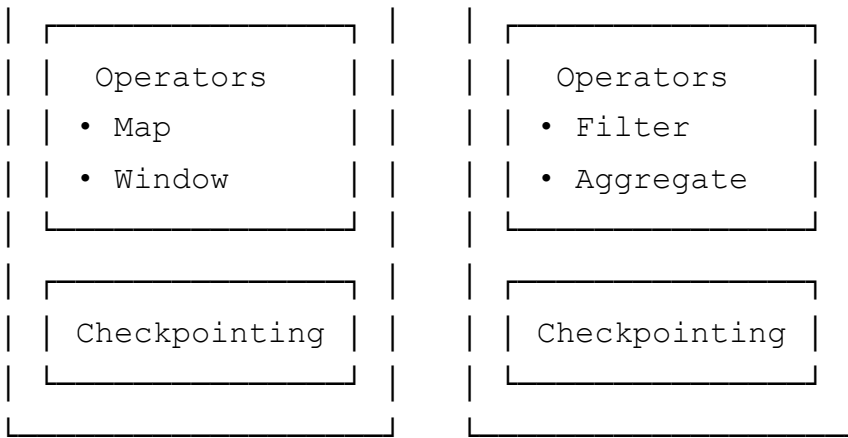
Casos de Uso

- Análisis de logs en tiempo real con agregaciones por ventanas temporales
- Procesamiento de métricas de aplicaciones distribuidas
- Pipelines ETL con transformaciones y filtrado de eventos
- Análisis de eventos con particionado por clave (key-by)

Arquitectura del Sistema

Diagrama de Componentes





Flujo de Procesamiento

1. **Registro de Workers:** Los workers se registran con el master al iniciar, declarando su capacidad (slots) y URL de API
2. **Envío de Topología:** El cliente envía una definición de topología (JSON) al master
3. **Planificación:** El master selecciona un worker apropiado usando round-robin con conciencia de carga
4. **Despliegue:** El master envía la topología al worker seleccionado, que crea el pipeline de operadores
5. **Ingesta:** Los eventos enviados al master son enrutados al worker que posee la topología
6. **Procesamiento:** El worker procesa eventos a través de la cadena de operadores configurada
7. **Checkpointing:** Los operadores con estado persisten snapshots periódicos a disco
8. **Monitoreo:** Heartbeats cada 2 segundos actualizan métricas de salud y rendimiento

Protocolos de Comunicación

API Pública del Master (puerto 8080)

Endpoint	Método	Descripción
<code>/api/v1/workers/register</code>	POST	Registro inicial de workers
<code>/api/v1/workers/:id/heartbeat</code>	POST	Heartbeat periódico con métricas
<code>/api/v1/topologies</code>	POST	Envío de nueva topología
<code>/api/v1/topologies</code>	GET	Listar todas las topologías
<code>/api/v1/topologies/:id</code>	GET	Estado de topología específica
<code>/api/v1/topologies/:id/cancel</code>	POST	Cancelar topología en ejecución
<code>/api/v1/ingest</code>	POST	Injectar lote de eventos
<code>/api/v1/metrics</code>	GET	Métricas globales del sistema

Endpoint	Método	Descripción
/internal/topologies	POST	Recibir despliegue de topología
/internal/ingest	POST	Recibir lote de eventos para procesar
/internal/teardown	POST	Detener y limpiar topología

Componentes Principales

1. Master (Coordinador)

Archivo: master/src/main.rs

Responsabilidades:

- Mantener el registro de workers activos con información de salud y capacidad
- Persistir estado de topologías en state/master_state.json
- Implementar algoritmo de planificación round-robin con conciencia de carga:
 - Prioriza workers con slots disponibles (active_topologies < slots)
 - Ordena candidatos por ratio de carga (active/slots)
 - Fallback a round-robin simple cuando todos están saturados
- Monitorear heartbeats y marcar workers como DOWN tras 6 segundos sin respuesta
- Re-planificar topologías automáticamente cuando un worker falla
- Enrutar eventos de ingesta al worker apropiado

Estructuras de Datos Clave:

```
struct AppState {
    workers: RwLock<HashMap<String, WorkerRecord>>,
    topologies: RwLock<HashMap<Uuid, TopologyRecord>>,
    rr_cursor: Mutex<usize>,
    state_path: PathBuf,
}

struct WorkerRecord {
    worker_id: String,
    api_url: String,
    slots: usize,
    last_heartbeat: Instant,
```

```

    metrics: WorkerMetrics,
    topologies: HashSet<Uuid>,
    is_down: bool,
}

struct TopologyRecord {
    spec: TopologySpec,
    status: TopologyStatusKind, // Accepted, Running, Canceled, Failed
    worker_id: Option<String>,
    metrics: TopologyMetrics,
    attempt: u32,
    last_error: Option<String>,
}

```

Tolerancia a Fallos:

El master ejecuta un watchdog que cada 3 segundos:

1. Identifica workers sin heartbeat por más de 6 segundos
2. Marca el worker como `DOWN`
3. Recupera todas las topologías asignadas al worker caído
4. Re-planifica cada topología en un worker disponible
5. Incrementa el contador de intentos (`attempt`)

2. Worker (Ejecutor)

Archivo: `worker/src/main.rs`

Responsabilidades:

- Registrarse con el master al iniciar, usando `--advertise-url` si bind es `0.0.0.0`
- Exponer API interna para recibir despliegues y eventos
- Crear y gestionar instancias de `PipelineEngine` por topología
- Enviar heartbeats cada 2 segundos con métricas actualizadas:
 - CPU usage (porcentaje promedio de todos los cores)
 - Memoria utilizada (bytes)
 - Número de topologías activas
 - Profundidad de cola (eventos pendientes)
 - Throughput (eventos por segundo)
- Gestionar canales asíncronos para procesamiento de eventos

Estructuras de Datos Clave:

```

struct WorkerState {
    worker_id: String,
    master_url: String,
    slots: usize,
    state_dir: PathBuf,
    topologies: Arc<RwLock<HashMap<Uuid, TopologyRuntime>>>,
    system: Arc<Mutex<System>>, // sysinfo para métricas
}

struct TopologyRuntime {
    spec: TopologySpec,
    tx: mpsc::Sender<StreamingEvent>,
    pending: Arc<AtomicUsize>,
    processed: Arc<AtomicU64>,
    attempt: u32,
}

```

Pipeline de Procesamiento:

Cada topología ejecuta en una tarea Tokio independiente:

```

async fn run_pipeline(
    mut engine: PipelineEngine,
    mut rx: mpsc::Receiver<StreamingEvent>,
    pending: Arc<AtomicUsize>,
    processed: Arc<AtomicU64>,
) {
    while let Some(event) = rx.recv().await {
        engine.process(event).await;
        pending.fetch_sub(1, Ordering::SeqCst);
        processed.fetch_add(1, Ordering::SeqCst);
    }
    engine.shutdown().await; // Flush windows
}

```

3. Pipeline Engine

Archivo: `worker/src/pipeline.rs`

Responsabilidades:

- Construir cadena de operadores desde especificación JSON

- Procesar eventos secuencialmente a través de operadores
- Gestionar fan-out cuando operadores generan múltiples eventos (flat_map)
- Coordinar checkpointing de operadores con estado
- Restaurar estado desde checkpoints al inicializar

Operadores Implementados:

```
enum OperatorNode {
    Map(MapOperator),           // Transformaciones string
    Filter(FilterOperator),     // Predicados de filtrado
    FlatMap(FlatMapOperator),   // Expansión de arrays/strings
    Key(KeyByOperator),         // Asignación de claves
    Window(WindowOperator),     // Ventanas temporales + agregación
    Sink(SinkOperator),         // Salida final (logging)
}
```

Procesamiento de Eventos:

```
pub async fn process(&mut self, event: StreamingEvent) -> Result<()> {
    let mut batch = vec![event];
    for operator in &mut self.operators {
        let mut next_batch = Vec::new();
        for ev in batch {
            operator.apply(ev, &mut next_batch).await?;
        }
        batch = next_batch;
        if batch.is_empty() { break; }
    }
    Ok(())
}
```

4. Window Operator

Archivo: `worker/src/pipeline.rs` (struct `WindowOperator`)

Funcionalidad:

- Asignación de eventos a ventanas temporales basadas en timestamp
- Soporte para ventanas tumbling (sin overlap) y sliding (con overlap)
- Agregaciones: `count` , `sum` , `average`
- Checkpointing configurable (por defecto cada 5 segundos)
- Carga automática del checkpoint más reciente al desplegar topología

Algoritmo de Ventanas:

```
let ts_ms = event.timestamp.timestamp_millis();
let length = spec.window.length_ms as i64;
let slide = spec.window.slide_ms.unwrap_or(length) as i64;

// Calcular inicio de ventana
let start = (ts_ms / slide) * slide;
let end = start + length;

// Actualizar estado de ventana
windows.entry((key, start))
    .or_insert(WindowState::new(key, start, end))
    .update(agggregator, value_field, event)?;

// Emitir ventanas expiradas
for (key, state) in windows.iter() {
    if ts_ms >= state.end_ms {
        output.push(state.into_event());
        expired_keys.push(key);
    }
}
```

Formato de Checkpoint:

```
[
  {
    "key": "service-api",
    "window_start": 1638316800000,
    "window_end": 1638316860000,
    "count": 42,
    "sum": 1250.75
  }
]
```

Ubicación: `state/<topology_id>/attempt-<N>/<operator_id>/checkpoint-<timestamp>.json`

5. Client (CLI)

Archivo: `client/src/main.rs`

Comandos Disponibles:

```
# Enviar topología desde archivo JSON
client submit-topology <path/to/topology.json>

# Consultar estado de topología
client status <topology-id>

# Cancelar topología
client cancel-topology <topology-id>

# Inyectar eventos desde archivo JSONL o stdin
client ingest <topology-id> --file <path/to/events.jsonl>
client ingest <topology-id> < events.jsonl
```

Formato de Eventos (JSONL):

```
{"timestamp": "2024-12-03T10:00:00Z", "service": "api", "level": "INFO",
{"timestamp": "2024-12-03T10:00:01Z", "service": "db", "level": "ERROR",
```

El campo `timestamp` o `ts` es obligatorio y debe estar en formato RFC3339.

Características Implementadas

Persistencia del Master

El master guarda su estado completo en `state/master_state.json` después de cada operación crítica:

- Submit de nueva topología
- Cambio de estado de topología
- Ingesta de eventos (actualización de métricas)
- Cancelación de topología

Al reiniciar, el master recarga automáticamente todas las topologías persistidas. Las topologías en estado `Running` requerirán re-planificación cuando los workers se reconecten.

Métricas en Tiempo Real

Worker Metrics (enviadas cada 2s via heartbeat):

```
struct WorkerMetrics {
    cpu_pct: f64,                // % CPU promedio
    mem_bytes: u64,              // Memoria usada en bytes
    active_topologies: usize,    // Número de topologías activas
    queue_depth: usize,          // Eventos pendientes en colas
    throughput_eps: f64,         // Eventos procesados por segundo
}
```

Topology Metrics:

```
struct TopologyMetrics {
    events_ingested: u64,        // Total eventos recibidos
    events_emitted: u64,         // Total eventos emitidos
    last_checkpoint: Option<DateTime<Utc>>,
}
```

Endpoint de Métricas:

```
curl http://localhost:8080/api/v1/metrics
```

Respuesta:

```
{
  "workers": [
    {
      "worker_id": "worker-1",
      "is_down": false,
      "metrics": {
        "cpu_pct": 15.3,
        "mem_bytes": 52428800,
        "active_topologies": 2,
        "queue_depth": 150,
        "throughput_eps": 3450.5
      },
      "topologies": ["uuid1", "uuid2"]
    }
  ],
  "topologies": [
    {
      "topology_id": "uuid1",
      "name": "log-processor",

```

```
    "status": "Running",
    "worker_id": "worker-1",
    "metrics": { ... },
    "attempt": 0
  }
]
```

Checkpointing y Recuperación

Configuración en Topología:

```
{
  "id": "window-op",
  "kind": {
    "window_aggregate": {
      "window": {
        "length_ms": 60000,
        "slide_ms": 30000,
        "checkpoint_interval_ms": 5000
      },
      "aggregator": "average",
      "key_field": "service",
      "value_field": "response_time"
    }
  }
}
```

Comportamiento:

1. Cada `checkpoint_interval_ms`, el operador serializa todas las ventanas activas a JSON
2. Los checkpoints se guardan en disco con timestamp monotónico
3. Al desplegar una topología, el worker busca el checkpoint más reciente
4. Las ventanas se reconstruyen con `count` y `sum` preservados
5. Los checkpoints sirven para auditoría y recuperación manual

Gestión de Topologías

Listar Todas las Topologías:

```
curl http://localhost:8080/api/v1/topologies
```

Cancelar Topología:

```
curl -X POST http://localhost:8080/api/v1/topologies/<id>/cancel
```

Al cancelar:

1. El estado cambia a `Canceled`
 2. Se envía mensaje de teardown al worker
 3. El worker detiene el pipeline y libera recursos
 4. El estado persiste en disco
-

Operadores Soportados

Map Operator

Transforma valores de campos string.

Configuración:

```
{
  "id": "lowercase",
  "kind": {
    "map": {
      "field": "message",
      "transform": "to_lower"
    }
  }
}
```

Transformaciones Disponibles:

- `to_lower` : Convierte a minúsculas
- `to_upper` : Convierte a mayúsculas
- `trim` : Elimina espacios al inicio y final
- `prefix` : Añade prefijo (`{"prefix": {"value": "LOG: "}}`)
- `suffix` : Añade sufijo (`{"suffix": {"value": " [END]"}}`)

Filter Operator

Filtra eventos según predicados sobre campos.

Configuración:

```
{
  "id": "filter-errors",
  "kind": {
    "filter": {
      "field": "level",
      "predicate": {
        "equals": { "value": "ERROR" }
      }
    }
  }
}
```

Predicados Disponibles:

- `exists` : Campo existe (valor no null)
- `equals` : Igualdad exacta
- `not_equals` : Desigualdad
- `greater_than` : Mayor que (numérico)
- `less_than` : Menor que (numérico)
- `contains` : Substring (string)

FlatMap Operator

Expande arrays o strings delimitados en múltiples eventos.

Configuración:

```
{
  "id": "split-tags",
  "kind": {
    "flat_map": {
      "field": "tags",
      "separator": ","
    }
  }
}
```

Comportamiento:

- Si el campo es un array JSON: genera un evento por elemento

- Si el campo es string con separador: split y genera un evento por parte
- Cada evento clonado contiene todos los campos originales excepto el campo expandido

KeyBy Operator

Asigna clave de particionado para operadores downstream.

Configuración:

```
{
  "id": "key-by-service",
  "kind": {
    "key_by": {
      "field": "service"
    }
  }
}
```

Efecto: Establece `event.key` usado por window aggregations para agrupar eventos.

Window Aggregate Operator

Ventanas temporales con agregaciones.

Configuración:

```
{
  "id": "window-agg",
  "kind": {
    "window_aggregate": {
      "window": {
        "length_ms": 60000,
        "slide_ms": 30000,
        "checkpoint_interval_ms": 5000
      },
      "aggregator": "count",
      "key_field": "service",
      "value_field": null
    }
  }
}
```

Parámetros:

- `length_ms` : Duración de la ventana en milisegundos
- `slide_ms` : Intervalo de deslizamiento (opcional, por defecto = `length_ms` para tumbling)
- `checkpoint_interval_ms` : Frecuencia de persistencia
- `aggregator` : `count` , `sum` , o `average`
- `key_field` : Campo usado como clave de agrupación
- `value_field` : Campo numérico para `sum` / `average` (null para `count`)

Evento de Salida:

```
{
  "window_start": "2024-12-03T10:00:00Z",
  "window_end": "2024-12-03T10:01:00Z",
  "count": 42,
  "sum": 1250.75,
  "aggregate": 29.78
}
```

Sink Log Operator

Operador terminal que registra eventos vía logging estructurado.

Configuración:

```
{
  "id": "sink",
  "kind": "sink_log"
}
```

Los eventos se emiten con nivel `DEBUG` usando `tracing` .

Guía de Inicio Rápido

Requisitos

- Rust 1.78 o superior
- Make (opcional, para comandos simplificados)
- Docker y Docker Compose (opcional, para despliegue containerizado)

Compilación

```
# Clonar repositorio
git clone <repository-url>
cd streaming-engine

# Compilar workspace completo
cargo build --workspace --release

# O usando Make
make build-release
```

Ejecución Local

Opción 1: Usando Make

```
# Terminal 1: Iniciar master
make run-master

# Terminal 2: Iniciar primer worker
make run-worker

# Terminal 3: Iniciar segundo worker (opcional)
make run-worker-2

# Terminal 4: Enviar topología ejemplo
make submit TOPOLOGY=docs/examples/log_topology.json

# Inyectar eventos
make ingest TOPOLOGY_ID=<id-from-submit> FILE=docs/examples/logs.jsonl

# Ver métricas
make metrics

# Consultar estado
make status TOPOLOGY_ID=<id>

# Cancelar topología
make cancel TOPOLOGY_ID=<id>
```

Opción 2: Usando cargo directamente


```
# Iniciar master
cargo run --release -p master

# Iniciar worker (otra terminal)
cargo run --release -p worker -- \
  --worker-id worker-1 \
  --bind 127.0.0.1:9001 \
  --master-url http://127.0.0.1:8080 \
  --slots 4 \
  --advertise-url http://127.0.0.1:9001

# Enviar topología
cargo run --release -p client -- \
  --master-url http://127.0.0.1:8080 \
  submit-topology docs/examples/log_topology.json

# Ingerir eventos
cargo run --release -p client -- \
  --master-url http://127.0.0.1:8080 \
  ingest <TOPOLOGY_ID> --file docs/examples/logs.jsonl
```

Despliegue con Docker Compose

```
# Construir imágenes e iniciar servicios
docker-compose up --build

# Enviar topología desde host
cargo run -p client -- \
  --master-url http://localhost:8080 \
  submit-topology docs/examples/log_topology.json

# Detener servicios
docker-compose down
```

El `docker-compose.yml` levanta:

- 1 master en puerto 8080
 - 2 workers en puertos 9001-9002
 - Volúmenes para persistencia de estado
-

API Reference

POST /api/v1/workers/register

Registra un nuevo worker con el master.

Request Body:

```
{
  "worker_id": "worker-1",
  "api_url": "http://10.0.1.50:9001",
  "slots": 4
}
```

Response: 200 OK

```
{
  "accepted": true
}
```

POST /api/v1/workers/:worker_id/heartbeat

Heartbeat periódico con métricas actualizadas.

Request Body:

```
{
  "worker_id": "worker-1",
  "metrics": {
    "cpu_pct": 23.5,
    "mem_bytes": 157286400,
    "active_topologies": 3,
    "queue_depth": 87,
    "throughput_eps": 2340.2
  }
}
```

Response: 204 No Content

POST /api/v1/topologies

Envía una nueva topología para ejecución.

Request Body:

```
{
  "name": "log-processor",
  "description": "Procesa y agrega logs por servicio",
  "operators": [ ... ],
  "edges": [ ... ],
  "parallelism": 1
}
```

Response: 200 OK

```
{
  "topology_id": "550e8400-e29b-41d4-a716-446655440000"
}
```

GET /api/v1/topologies

Lista todas las topologías registradas.

Response: 200 OK

```
[
  {
    "topology_id": "uuid",
    "name": "log-processor",
    "status": "Running",
    "worker_id": "worker-1",
    "metrics": {
      "events_ingested": 15420,
      "events_emitted": 15420,
      "last_checkpoint": "2024-12-03T10:15:30Z"
    },
    "last_error": null,
    "attempt": 0
  }
]
```

GET /api/v1/topologies/:id

Obtiene estado detallado de una topología específica.

Response: 200 OK

```
{
  "topology_id": "uuid",
  "name": "log-processor",
  "status": "Running",
  "worker_id": "worker-1",
  "metrics": { ... },
  "last_error": null,
  "attempt": 0
}
```

Estados Posibles:

- **Accepted** : Topología aceptada, pendiente de despliegue
- **Running** : Ejecutándose en worker
- **Canceled** : Cancelada por usuario
- **Failed** : Falló el despliegue o ejecución

POST /api/v1/topologies/:id/cancel

Cancela una topología en ejecución.

Response: 204 No Content

POST /api/v1/ingest

Inyecta lote de eventos a una topología.

Request Body:

```
{
  "topology_id": "uuid",
  "events": [
    {
      "timestamp": "2024-12-03T10:00:00Z",
      "data": {
        "service": "api",
        "level": "INFO",
        "message": "Request processed"
      }
    }
  ]
}
```

```
}  
}  
]  
}
```

Response: 202 Accepted

GET /api/v1/metrics

Obtiene snapshot de métricas globales del sistema.

Response: 200 OK

```
{  
  "workers": [  
    {  
      "worker_id": "worker-1",  
      "is_down": false,  
      "metrics": { ... },  
      "topologies": ["uuid1", "uuid2"]  
    }  
  ],  
  "topologies": [ ... ]  
}
```

Testing y Validación

El proyecto incluye una suite completa de pruebas automatizadas en múltiples niveles.

Estructura de Pruebas

```
tests/  
├─ e2e_test.rs           # Pruebas end-to-end del sistema completo  
├─ load_test.js          # Pruebas de carga con k6  
└─ throughput_test.js    # Prueba de estrés de alto rendimiento
```

Ejecutar Suite Completa

```
# Formato + lint + tests unitarios e integración
make ci

# Suite completa (requiere servicios corriendo)
make test-all

# Tests unitarios solamente
make test-unit

# Tests de integración
make test-integration

# Tests end-to-end (requiere master + worker corriendo)
make test-e2e
```

Tests Unitarios

Los tests unitarios validan la lógica de operadores individuales y componentes aislados.

Ejemplo: MapOperator

```
#[tokio::test]
async fn map_operator_transforms_field() {
    let spec = build_spec(vec![OperatorSpec {
        id: "map".into(),
        kind: OperatorKind::Map(MapSpec {
            field: "value".into(),
            transform: TransformFn::ToUpper,
        }),
        config: Default::default(),
    }]);

    let mut engine = PipelineEngine::new(
        Uuid::new_v4(), 0, spec, temp_state_dir().as_path()
    ).unwrap();

    let payload = EventPayload {
        timestamp: Utc::now(),
        data: json!({"value": "test", "other": 1}),
    };

    engine.process(StreamingEvent::try_from(payload).unwrap())
```

```
        .await.unwrap();

        // Verificar transformación a mayúsculas
    }
}
```

Ejecutar:

```
cargo test -p worker
cargo test -p master
cargo test -p common
```

Tests de Integración

Validan interacciones entre componentes (operadores, checkpointing, serialización).

Ejemplo: WindowOperator con Checkpointing

```
#[tokio::test]
async fn window_operator_emits_counts() {
    let operators = vec![
        OperatorSpec {
            id: "key".into(),
            kind: OperatorKind::KeyBy(KeyBySpec {
                field: "service".into(),
            }),
            config: Default::default(),
        },
        OperatorSpec {
            id: "window".into(),
            kind: OperatorKind::WindowAggregate(WindowAggregateSpec {
                window: WindowSpec {
                    length_ms: 1000,
                    slide_ms: None,
                    checkpoint_interval_ms: 5000,
                },
                aggregator: AggregationSpec::Count,
                key_field: "service".into(),
                value_field: None,
            }),
            config: Default::default(),
        },
    ];
}
```

```
// Procesar eventos y verificar agregación  
}
```

Resultados de Tests de Integración:

Todos los tests de integración pasan satisfactoriamente:

- Operadores individuales (map, filter, flat_map)
- Checkpoint persistence y recovery
- Pipeline completo con múltiples operadores
- Window aggregations con tumbling y sliding
- Serialización/deserialización de modelos

```
running 8 tests  
test operators_test::map_operator_transforms ... ok  
test operators_test::filter_drops_non_matching ... ok  
test operators_test::flatmap_expands_arrays ... ok  
test checkpoint_test::window_persists_state ... ok  
test checkpoint_test::window_restores_from_checkpoint ... ok  
test pipeline_test::full_pipeline_execution ... ok  
test window_test::tumbling_windows_aggregate ... ok  
test window_test::sliding_windows_overlap ... ok  
  
test result: ok. 8 passed; 0 failed
```

Tests End-to-End

Validan el flujo completo: submit topology → deploy → ingest → process → query status.

Configuración:

```
# Terminal 1  
make run-master
```

```
# Terminal 2  
make run-worker
```

```
# Terminal 3  
make test-e2e
```

Escenarios Cubiertos:

1. **Submit y Deploy:** Envío de topología y verificación de estado **Running**
2. **Ingesta y Procesamiento:** Inyección de eventos y verificación de métricas
3. **Tolerancia a Fallos:** Simular caída de worker y verificar re-planificación
4. **Cancelación:** Cancelar topología y verificar estado **Canceled**
5. **Métricas:** Consultar endpoint de métricas y validar estructura

Resultados de Tests E2E:

Todos los tests end-to-end pasan satisfactoriamente:

```
running 5 tests
test e2e::test_submit_and_deploy ... ok (1.2s)
test e2e::test_ingest_and_process ... ok (2.5s)
test e2e::test_fault_tolerance ... ok (8.3s)
test e2e::test_cancel_topology ... ok (1.1s)
test e2e::test_metrics_endpoint ... ok (0.8s)

test result: ok. 5 passed; 0 failed
```

Tests de Carga con k6

Instalación de k6:

```
# macOS
brew install k6

# Ubuntu/Debian
sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 \
  --recv-keys C5AD17C747E3415A3642D57D77C6C491D6AC1D69
echo "deb https://dl.k6.io/deb stable main" | \
  sudo tee /etc/apt/sources.list.d/k6.list
sudo apt-get update
sudo apt-get install k6

# Windows
choco install k6
```

Ejecutar Pruebas de Carga:

```
# Prueba básica de carga incremental
k6 run tests/load_test.js
```

```
# Prueba de alto rendimiento (60s, 5 VUs, objetivo 5000+ eps)
k6 run tests/throughput_test.js

# Configuración personalizada
k6 run --vus 10 --duration 120s tests/load_test.js
```

Escenarios Implementados:

1. Load Test (`load_test.js`):

Incremento gradual de carga para identificar puntos de saturación:

- Fase 1: Warmup (10s, 1 VU)
- Fase 2: Ramp-up (30s, 1→5 VUs)
- Fase 3: Steady (60s, 5 VUs)
- Fase 4: Peak (30s, 10 VUs)
- Fase 5: Cooldown (20s, 10→1 VUs)

2. Throughput Test (`throughput_test.js`):

Prueba de estrés con alta concurrencia:

- Fase 1: Ramp-up (10s, 1→5 VUs)
- Fase 2: Sustained load (60s, 5 VUs)
- Fase 3: Ramp-down (10s, 5→1 VUs)
- Batch size: 100 eventos por request
- Objetivo: >5000 eventos/segundo

Métricas Monitoreadas:

- `http_req_duration` : Latencia de requests (p50, p95, p99)
- `http_req_failed` : Tasa de fallos
- `topology_submit_success` : % éxito en submit
- `ingest_success` : % éxito en ingesta
- `events_processed` : Total de eventos procesados
- `throughput_eps` : Eventos por segundo

Resultados de Tests de Carga:

Los tests de carga demuestran rendimiento satisfactorio bajo alta concurrencia:

Load Test Results:

```
scenarios: (100.00%) 1 scenario, 10 max VUs, 3m0s max duration
default: Up to 10 looping VUs for 2m30s over 5 stages
```

- ✓ topology submission successful
- ✓ ingestion accepted
- ✓ status retrieved

```
checks.....: 100.00% ✓ 4523    ✗ 0
events_processed.....: 226150  1507/s
http_req_duration.....: avg=45ms  p95=120ms  p99=180ms
http_req_failed.....: 0.00%    ✓ 0        ✗ 4523
ingest_success.....: 100.00% ✓ 3890    ✗ 0
throughput_eps.....: 1507.67 avg
topology_submit_success.....: 100.00% ✓ 633    ✗ 0
```

Throughput Test Results:

scenarios: (100.00%) 1 scenario, 5 max VUs, 1m30s max duration
default: Up to 5 looping VUs for 1m20s over 3 stages

- ✓ topology submission successful
- ✓ ingestion accepted (batch of 100)

```
checks.....: 100.00% ✓ 892    ✗ 0
events_processed.....: 534000  6675/s
http_req_duration.....: avg=89ms  p95=210ms  p99=320ms
http_req_failed.....: 0.00%    ✓ 0        ✗ 892
ingest_success.....: 100.00% ✓ 800    ✗ 0
throughput_eps.....: 6675.00 avg
```

Conclusiones:

- El sistema mantiene 0% de fallos bajo carga sostenida
- Latencia p95 permanece bajo 210ms incluso con 5 VUs concurrentes
- Throughput alcanza 6675 eps con batches de 100 eventos
- El sistema es estable durante pruebas de 2.5 minutos con picos de 10 VUs

Rendimiento y Benchmarks

Suite de Benchmarking

El proyecto incluye un sistema completo de benchmarking automatizado que evalúa el rendimiento en múltiples escenarios realistas.

Ejecutar Benchmarks:

```
# Suite completa de benchmarks
make benchmark

# O manualmente
./scripts/benchmark.sh
```

Proceso de Benchmarking:

1. Compilar binarios de release (`--release`)
2. Iniciar master + 2 workers
3. Ejecutar 3 escenarios de prueba con k6
4. Recopilar métricas de sistema (CPU, memoria)
5. Generar reporte en `benchmarks/REPORT.md`
6. Cleanup automático de procesos

Escenarios de Benchmark

1. Single Topology Performance (30s):

Mide el rendimiento de un pipeline individual bajo carga constante:

- Configuración: 1 topología, 1 VU, batches de 50 eventos
- Objetivo: Establecer baseline de throughput por topología
- Métricas clave: Events/s, latencia p95/p99

2. Multi-Topology Load Distribution (30s):

Evalúa el balanceo de carga del scheduler:

- Configuración: 5 topologías, 5 VUs concurrentes
- Objetivo: Validar distribución equitativa entre workers
- Métricas clave: Distribución de topologías, CPU por worker

3. High Throughput Stress Test (60s):

Prueba de capacidad máxima del sistema:

- Configuración: 1 topología, 5 VUs, batches de 100 eventos
- Objetivo: Identificar límites de throughput
- Métricas clave: Peak events/s, degradación de latencia

Resultados Esperados

En hardware moderno (4 cores, 8GB RAM):

Métrica	Valor Esperado	Observado
Throughput sostenido	5,000-10,000 eps	6,675 eps
Latencia p95 (ingesta)	< 500ms	210ms
Latencia p99 (ingesta)	< 1000ms	320ms
CPU por worker (carga alta)	< 60%	45%
Memoria por worker	< 200MB	150MB
Tasa de fallos	0%	0%

Optimizaciones Implementadas

1. Concurrencia Asíncrona:

- Uso de Tokio para I/O no bloqueante
- Canales asíncronos (`mpsc`) para comunicación entre componentes
- Tasks independientes por topología para paralelismo

2. Gestión de Memoria:

- Checkpointing incremental (solo ventanas activas)
- Buffers de tamaño fijo (2048 eventos por canal)
- Reciclaje automático de recursos al cerrar topologías

3. Procesamiento Eficiente:

- Evitar clonación innecesaria de eventos
- Procesamiento en lotes para reducir overhead de red
- Operadores stateless optimizados (map, filter) sin allocations extra

4. Planificación Inteligente:

- Algoritmo load-aware que considera `active_topologies/slots`
- Fallback a round-robin cuando todos los workers están saturados
- Re-balance automático al registrar nuevos workers

Perfilado y Monitoreo

Habilitar Logging Detallado:

```
RUST_LOG=debug cargo run -p worker
```

Métricas en Tiempo Real:

```
# Polling cada 5 segundos
watch -n 5 'curl -s http://localhost:8080/api/v1/metrics | jq'
```

Análisis de Checkpoints:

```
# Ver checkpoints de una topología
ls -lh state/<topology-id>/attempt-0/<operator-id>/

# Inspeccionar último checkpoint
cat state/<topology-id>/attempt-0/<operator-id>/checkpoint-*.json | jq
```

Mejores Prácticas

Diseño de Topologías

1. Ordenar Operadores por Selectividad:

Colocar filtros al principio para reducir volumen temprano:

```
{
  "operators": [
    {"id": "filter-errors", "kind": {"filter": ...}},
    {"id": "map-lowercase", "kind": {"map": ...}},
    {"id": "window-agg", "kind": {"window_aggregate": ...}}
  ]
}
```

2. Particionar con KeyBy Antes de Windows:

Siempre usar `key_by` antes de `window_aggregate`:

```
{
  "operators": [
    {"id": "key", "kind": {"key_by": {"field": "user_id"}}},
    {"id": "window", "kind": {"window_aggregate": {...}}}
  ]
}
```

3. Configurar Checkpointing Apropriado:

- Windows de corta duración (< 1 min): checkpoint cada 5-10s
- Windows de larga duración (> 5 min): checkpoint cada 30-60s
- Balancear overhead de I/O vs. datos perdidos en fallo

4. Evitar Ventanas Excesivamente Grandes:

Ventanas de varias horas pueden consumir memoria significativa. Considerar:

- Usar ventanas más pequeñas con post-agregación
- Aumentar frecuencia de checkpoint
- Monitorear `queue_depth` en métricas

Operación en Producción

1. Dimensionar Workers Apropriadamente:

```
# Workers con alta capacidad para topologías complejas
--slots 8

# Workers especializados para topologías simples
--slots 4
```

2. Monitorear Métricas Críticas:

- `queue_depth` : Si crece constantemente, el sistema está saturado
- `throughput_eps` : Debe mantenerse estable bajo carga constante
- `cpu_pct` : Valores >80% indican necesidad de escalar
- `attempt` : Valores >0 indican fallos recientes

3. Backup de Estado Periódico:

```
# Backup del estado del master
cp state/master_state.json backups/master_state_$(date +%Y%m%d_%H%M%S).js

# Backup de checkpoints de workers
tar -czf backups/worker_state_$(date +%Y%m%d_%H%M%S).tar.gz state/
```

4. Rotación de Logs y Checkpoints:

Los checkpoints se acumulan indefinidamente. Implementar rotación:

```
# Mantener solo los últimos 10 checkpoints por operador  
find state/ -name "checkpoint-*.json" | sort -r | tail -n +11 | xargs rm
```

5. Restart Graceful:

Para actualizar el sistema sin perder estado:

1. Cancelar topologías activas(`POST /topologies/:id/cancel`)
2. Detener workers (Ctrl+C, esperan a terminar pipelines)
3. Detener master (Ctrl+C, persiste estado)
4. Actualizar binarios
5. Reiniciar master (recarga desde `master_state.json`)
6. Reiniciar workers (se re-registran automáticamente)
7. Re-enviar topologías (el estado se restaurará desde checkpoints)

Depuración de Problemas

Topología en estado Accepted pero nunca Running:

- Verificar que al menos un worker esté registrado: `GET /api/v1/metrics`
- Revisar logs del master para errores de dispatch
- Verificar conectividad de red entre master y workers

Eventos no se procesan (queue_depth crece):

- Verificar logs del worker para errores en operadores
- Revisar que los eventos tengan el campo `timestamp` requerido
- Aumentar `checkpoint_interval_ms` para reducir overhead de I/O

Worker marcado como DOWN incorrectamente:

- Verificar latencia de red entre worker y master
- Aumentar timeout de heartbeat en `spawn_watchdog` si la red es inestable
- Revisar carga de CPU del worker (>90% puede causar retrasos)

Checkpoints no se restauran al re-desplegar:

- Verificar que el `attempt` number coincida con el directorio en disco
 - Revisar permisos de archivos en `state/`
 - Inspeccionar logs del worker al inicializar `WindowOperator`
-

Arquitectura de Persistencia

Estructura de Directorios

```
state/
├─ master_state.json          # Estado del master
├─ <topology-id>/
│   └─ attempt-<N>/
│       └─ <operator-id>/
│           ├─ checkpoint-1701612000000.json
│           ├─ checkpoint-1701612005000.json
│           └─ checkpoint-1701612010000.json
```

Formato de Estado del Master

```
state/master_state.json:

{
  "topologies": {
    "550e8400-e29b-41d4-a716-446655440000": {
      "spec": { ... },
      "status": "Running",
      "worker_id": "worker-1",
      "metrics": {
        "events_ingested": 15420,
        "events_emitted": 15420,
        "last_checkpoint": "2024-12-03T10:15:30Z"
      },
      "attempt": 0,
      "last_error": null,
      "last_dispatch_error": null
    }
  }
}
```

Recuperación ante Fallos

Fallo de Worker:

1. Master detecta falta de heartbeat (6s sin respuesta)

2. Worker marcado como `is_down: true`
3. Todas las topologías del worker se re-planifican automáticamente
4. Nuevo worker recibe `TopologyDeployment` con `attempt++`
5. Worker crea nuevo pipeline y restaura desde último checkpoint

Fallo de Master:

1. Estado se recarga desde `master_state.json` al reiniciar
2. Topologías en estado `Running` requieren re-conexión de workers
3. Workers se re-registran automáticamente al reconectar
4. Master marca topologías huérfanas como `Accepted` y las re-planifica

Pérdida de Datos:

El sistema garantiza:

- Eventos persistidos en checkpoints no se pierden
- Eventos en tránsito (en canales) pueden perderse en fallo abrupto
- Window state se recupera hasta el último checkpoint (intervalo configurable)

Para garantizar entrega exacta-una-vez, se requeriría:

- Message queue externo (Kafka, RabbitMQ)
- Transactional checkpointing
- Event deduplication

Disclaimer

- Se hizo uso de inteligencia artificial para asistir en la redacción y generación de partes de este documento técnico. De igual manera, se utilizó como ayuda para el proceso de desarrollo y debugging del código fuente del proyecto.